

Evaluation Algorithmique et Programmation Parallèle

Durée de l'épreuve 1h30.

Question de cours

- ✓1) Quel est le principal intérêt des communications point-à-point non-bloquantes MPI ?
- ✓2) Qu'est-ce qu'un type dérivé (donner des exemples) ?
- ✓3) Que sont les communications point à point ?
- ✓4) Que sont les communications collectives ?

Exercice 1

Pour chacun des extraits de programme suivants analyser le code, décrire le problème et proposer une solution.

Question 1

Dans ce qui suit, on désigne par rang le rang de la tâche MPI, et par P la taille du communicateur MPI_COMM_WORLD.

```
1 : MPI_Request req ;
2 : MPI_Status stal, sta2 ;
3 : double m = 1.0 / (rang+1) ;
4 : int v = 2 ;
5 :
6 : if ( rang == 0 || rang == 2 ) {
7 :   if ( rang == 2 ) v = 0 ;
8 :   MPI_Isend(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &req) ;
9 :   MPI_Recv(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &stal) ;
10:   MPI_Wait(&req, &sta2) ;
11: }
```

Question 2 ✓

Dans ce qui suit, on désigne par rang le rang de la tâche MPI, et par P la taille du communicateur MPI_COMM_WORLD.

```
1 : MPI_Status sta ;
2 : int l, r, w ;
3 : w = rang ;
4 : l = (rang+P-1)%P ;
5 : r = (rang+1)%P ;
6 : MPI_Send(&w, 1, MPI_INT, l, 0, MPI_COMM_WORLD) ;
7 : MPI_Recv(&w, 1, MPI_INT, r, 0, MPI_COMM_WORLD, &sta) ;
```

Question 3 ✓

Dans ce qui suit, on désigne par rang le rang de la tâche MPI, et par P la taille du communicateur MPI_COMM_WORLD.

```
1 : double som_glob, som_loc = 0 ;
2 : /* x est un tableau distribué de réels (double précision)
3 :    N est le nombre d'éléments associé à la tâche
4 : */
```

double som_glob, som_loc = 0.

```
5 : for ( i = 0 ; i < N ; i++ ) {
6 :   som_loc += x[i] ;
7 : }
8 : MPI_Reduce( &som_loc, &som_glob, 1, MPI_DOUBLE, MPI_SUM, rang,
9 :   MPI_COMM_WORLD ) ;
10: if ( rang == 0 ) {
11:   printf( "Somme globale : %g\n", som_glob ) ;
12: }
```

Exercice 2 : ✓

Proposer un algorithme réalisant un échange de messages MPI sous la forme d'un anneau via des communications de type MPI_Send/MPI_Recv.

Exercice 3 : ✓

Le programme suivant permet de calculer un produit scalaire entre deux vecteurs avec un nombre N de composantes. Cette petite application est séquentielle. Le but de cet exercice est de paralléliser ce code avec plusieurs modèles de programmation parallèle.

```
1 : double * x ;
2 : double * y ;
3 : int N ;
4 : double final_res = 0.0;
5 : double compute( double * x, double * y, int N ) {
6 :   double res = 0.0;
7 :   int i ;
8 :   for ( i = 0 ; i < N ; i++ ) {
9 :     res += x[i] * y[i] ;
10:  }
11:  return res ;
12: }
13:
14: int main( int argc, char ** argv ) {
15:
16:   /* La fonction initialize_N permet d'initialiser la valeur de N */
17:   N = initialize_N() ;
18:
19:   /* Allocation des vecteurs x et y de taille N */
20:   x = (double *)malloc( N * sizeof( double ) ) ;
21:   y = (double *)malloc( N * sizeof( double ) ) ;
22:
23:   /* Initialisation des vecteurs x et y */
24:   /* La fonction read_vectors remplit les tableaux x */
25:   /* et y avec N valeurs (de l'index 0 à l'index N-1) */
26:   read_vectors(x, y, N) ;
27:
28:   /* Calcul du produit scalaire */
29:   final_res = compute(x, y, N);
30:
31:   printf( "Dot product with %d coordinates: %g\n", N, final_res ) ;
32: }
```

Question 1

Décrire brièvement où se trouve le parallélisme potentiel dans l'application séquentielle ci-dessus.

Question 2

Ecrire la fonction `main` avec un parallélisme à mémoire distribuée avec le modèle MPI (il est interdit de modifier les appels aux fonctions `initialize_N` et `read_vectors`).

Attention, ne pas oublier les étapes suivantes :

- Initialisation/terminaison du modèle MPI (on admettra que N est un multiple du nombre de tâches MPI)
- Distribution des tableaux x et y sur les différentes tâches MPI après initialisation des tableaux avec la fonction `read_vectors` appelée par une seule tâche MPI.
- Affichage du résultat final sur la tâche de rang 0

Algorithmique et Programmation Parallèle
Évaluation du 15/03/2024 – Ce document contient 4 pages
1h30 min – Tout document autorisé (non numérique)

Exercice I : Petit programme MPI

Soit la section de code suivante exécutée par tous les processus d'un programme MPI (il y'a au moins 2 processus, on suppose le programme déjà initialisé avec MPI_Init) :

```
static int val ;
int rang;

MPI_Comm_rank(MPI_COMM_WORLD, &rang) ;

val = 1;
if (rang == 0)
    val *= 10;

MPI_Barrier(MPI_COMM_WORLD) ;

val *= 2 ;
if (rang == 1)
    val *= 100 ;

MPI_Barrier(MPI_COMM_WORLD) ;
```

proc 0 = 10
proc 1 = 2 = 100
proc n = 2

Question 1 :

Pourrait-on supprimer les barrières MPI sans influencer le résultat du programme ? (justifiez brièvement)

Question 2 :

Suite à l'exécution de ces lignes, quelle est la valeur de la variable val en fonction du rang MPI ? (justifiez brièvement)

Exercice II : Implémentation d'un gather

Soit la fonction :

```
void allgather( int in, int *out ) ;
```

appelée en même temps par tous les processus MPI.

Elle collecte dans le tableau out le contenu de toutes les variables in de tous les processus MPI. Tous les processus doivent récupérer les résultats. Le tableau out est préalablement alloué au nombre total de processus.

La fonction allgather est implémentée dans le listing suivant, ainsi qu'un programme principal qui utilise cette fonction. Cependant le listing n'est pas complet.

Travail à faire : compléter les lignes */* A COMPLETER */* pour que la fonction allgather et le programme principal soient corrects.

Listing de la fonction allgather et son programme principal : complétez-le.

```

1. void allgather( int in, int *out ) {
2.     int rang, P;
3.
4.     /* A COMPLETER : il faut affecter le nombre de processus dans P */
5.     /* A COMPLETER : il faut affecter la variable rang */
6.
7.     if (rang == 0) {
8.         MPI_Status sta;
9.         out[0] = in;
10.        for(int p = 1 ; p < P ; p++) {
11.            int val;
12.            /* A COMPLETER : réception dans val de la valeur envoyée par le processus de rang p */
13.            /* A COMPLETER : mise à jour du tableau out */
14.        }
15.    } else {
16.        /* A COMPLETER : envoi de in au processus 0 */
17.    }
18.    /* A COMPLETER : le processus 0 diffuse out aux autres processus */
19.}
20.
21.int main(int argc, char *argv[])
22.{
23.    int rang, P, *out;
24.
25.    /* A COMPLETER */
26.    /* A COMPLETER : il faut affecter le nombre de processus dans P */
27.    /* A COMPLETER : il faut affecter la variable rang */
28.
29.    out = (int*)malloc(P*sizeof(int));
30.    allgather(rang, out);
31.    for(int p = 0 ; p < P ; p++)
32.        printf("P%d, out[%d] = %d\n", rang, p, out[p]);
33.    free(out);
34.
35.    /* A COMPLETER */
36.    return 0;
37.}

```

Exercice III : Calcul d'une série

Soit S_N une série où N est un entier >0 :

$$S_N = \frac{1}{N} \times \sum_{i=1}^N \frac{u_i}{1+u_i^2}$$

La suite u_i se définit comme suit :

$$\beta = 1.4, u_0 = \frac{-\beta}{2}$$

$$i \geq 1, u_i = u_{i-1} + \beta$$

ce qui peut encore se réécrire :

$$i \geq 1, u_i = u_0 + \beta \times i$$

Soit le programme séquentiel suivant qui calcule dans S la valeur S_N :

```
1 - void main ()
2 - {
3 -   double u, S, beta, sum = 0.0;
4 -   int i, N = 100000;
5 -
6 -   beta = 1.4;
7 -   u = -0.5*beta;
8 -
9 -   for (i=1; i<= N; i++) {
10 -    u += beta;
11 -    sum = sum + u/(1.0+u*u);
12 -  }
13 -
14 -   S = sum/(double)N;
15 -   printf("\n S = %lf\n ", S);
16 - }
```

Travail à faire :

Paralléliser le calcul de S avec OpenMP

Exercice IV : Boucles for

Soit le programme OpenMP suivant :

```
void main() {
    // (A)
    #pragma omp parallel num_threads(2)
    for(int i=0 ; i<10 ; i++) {
        printf("thr=%d, i=%d\n", omp_get_thread_num(), i);
    }

    // (B)
    #pragma omp for schedule(static)
    for(int j=0 ; j<10 ; j++) {
        printf("thr=%d, j=%d\n", omp_get_thread_num(), j);
    }

    // (C)
    #pragma omp parallel for schedule(static) num_threads(2)
    for(int k=0 ; k<10 ; k++) {
        printf("thr=%d, k=%d\n", omp_get_thread_num(), k);
    }
}
```

Question 1 :

Donnez pour chacune des boucles (A), (B) et (C), le nombre maximal de threads les exécutant. Justifiez brièvement.

Question 2 :

On fait l'hypothèse suivante : chacune des boucles est exécutée par son nombre maximal possible de threads (déterminé en Question 1).

Pour chacune des sections (A), (B) et (C), donnez les intervalles des valeurs de i (respectivement j et k) en fonction des numéros des threads.

Exercice V : Sémaphore

Soit le programme OpenMP suivant exécuté avec la variable d'environnement `OMP_NUM_THREADS` affectée à 4.

On fait l'hypothèse que les threads OpenMP de la région parallèle sont compatibles POSIX. Par conséquent, il est possible de faire appel à la bibliothèque *pthread* dans la région parallèle OpenMP.

```
1 - #include <stdio.h>
2 - #include <omp.h>
3 - #include <semaphore.h>
4 -
5 - #define VINI 2
6 -
7 - void main()
8 - {
9 -     sem_t sem;
10 -    sem_t *sem_ptr = &sem;
11 -    sem_init(&sem, 0, VINI);
12 -
13 -    #pragma omp parallel
14 -    {
15 -        int nt = omp_get_num_threads();
16 -
17 -        sem_wait(sem_ptr);
18 -        {
19 -            int myrank = omp_get_thread_num();
20 -            printf("%d\n", myrank);
21 -        }
22 -        sem_post(sem_ptr);
23 -    }
24 - }
```

2
∞
2

Questions 1-*) :

- 1-a) La variable `sem_ptr` est-elle locale à chaque thread de la région parallèle ou bien partagée entre ses threads ? Justifiez
- 1-b) La variable `nt` est-elle locale à chaque thread de la région parallèle ou bien partagée entre ses threads ? Justifiez
- 1-c) La valeur de `nt` est-elle identique à tous les threads ou bien différente par thread ? Justifiez
- 1-d) Combien de threads au plus vont exécuter `printf("%d\n", myrank)` (l. 20) ? Justifiez
- 1-e) L'ordre d'affichage des valeurs de `myrank` est-il identique d'une exécution à une autre ? Justifiez

Pour les questions suivantes, on fait l'hypothèse que `VINI` est définie à 1. La ligne 5 est remplacée par :

```
5 - #define VINI 1
```

Question 2 : Combien de threads au plus vont exécuter `printf("%d\n", myrank)` (l. 20) ? Justifiez

Question 3 : On désire avoir le même comportement du programme (avec `VINI 1`) sans utilisation du sémaphore.

Réécrivez le programme en remplaçant le sémaphore par une autre construction *pthread* (tout en conservant la région parallèle OpenMP).

Question 4 : On désire avoir le même comportement du programme (avec `VINI 1`) sans aucune fonction *pthread*.

Réécrivez le programme en utilisant uniquement OpenMP.